

BUSINESS REQUIREMENTS DOCUMENT

Reducing UPI Mandate Failures Through Behavioural Analysis

A Personal Case Study — Improving Recurring Payment Onboarding via Data-Driven Product Interventions

Prepared By	Preeti Jadhav
Date	02 Feb 2026
Domain	Payments / UPI Autopay
Project Type	Personal Case Study
Version	1.0

1. About This Document

This BRD is a personal case study I put together to practise writing product requirements using a real-world payments problem. The starting point was a dataset of 2,000 synthetic UPI mandate attempts, modelled on RBI-reported failure trends, which I used to identify the main failure drivers and translate them into product recommendations.

The document covers the problem background, analysis findings, functional and non-functional requirements, and a full set of user stories, structured the way I would present them in a working product team. The goal was to go from raw data insights to a proper requirements artefact, the way a BA would hand it off to engineering and design.

Field	Details
Prepared By	Preeti Jadhav
Domain	Payments / UPI Autopay
Project Type	Personal Case Study
Dataset	2,000-record synthetic dataset modelled on RBI failure trends

2. Executive Summary

UPI Autopay mandates power subscription billing, loan EMIs, insurance premiums, and other recurring payment use cases across India's digital payments ecosystem. Despite how widely they are used, industry data and the analysis for this project suggest that over 50% of mandate setup attempts fail, creating real friction right at the start of the user's journey.

The analysis pointed to three main drivers of failure, each needing a different kind of fix:

- Bank timeout errors (34% of failures) — a system-side problem that retry logic can address
- PIN entry errors (33%) — a user-side problem where better error messaging helps
- Invalid VPA entries (33%) — another user-side issue, fixable with real-time format validation

On top of that, there is a clear time-of-day pattern. Success rates nearly double in the evening (19:00–21:00), and Saturday at 20:00 reaches around 82% success, close to three times the rate of a weekday morning. That insight drives one of the key recommendations: guide users toward these higher-success windows instead of letting them retry randomly.

If even 50% of bank timeout failures are recovered through smart retry logic, the projected overall success rate improves from 42.35% to around 52%, roughly a 10 percentage point gain. That translates directly into more subscriptions activating, less drop-off at onboarding, and more reliable recurring payments.

3. Background & Problem Statement

3.1 What is UPI Autopay?

UPI Autopay is a recurring payment framework built on top of India's Unified Payments Interface (UPI), governed by NPCI and regulated by RBI. It allows users to pre-authorise recurring debits from their bank accounts, for things like Netflix subscriptions, SIP investments, insurance premiums, or loan EMIs.

The setup flow works like this: the user provides their VPA (Virtual Payment Address, e.g. name@okicici), confirms the mandate details, enters their UPI PIN to authorise the debit, and the request goes through the UPI rail to their bank. Each of those steps can go wrong.

3.2 The Problem

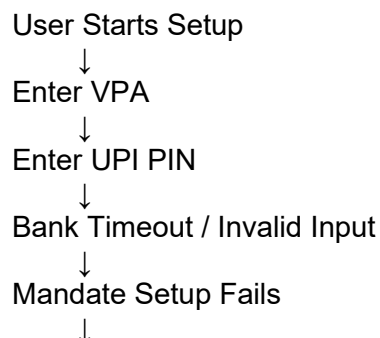
Based on the dataset and RBI industry benchmarks, 57.65% of mandate setup attempts fail. The breakdown of why they fail is fairly even across three categories:

Failure Reason	Type	Share of Failures	What it means
Bank Timeout	System	~34%	The bank's server didn't respond in time, usually due to infrastructure load
PIN Error	User Input	~33%	User entered the wrong UPI PIN, and there was no clear guidance on what to do next
Invalid VPA	User Input	~33%	User submitted a badly formatted or non-existent VPA, could have been caught earlier

The consequence is that a large share of users who try to set up a mandate give up mid-flow. That means lost subscriptions, incomplete onboarding, and a poor first impression of the recurring payments experience. The good news is that all three failure categories have practical product-level fixes.

3.3 Current vs Proposed User Flow

Current Flow:



User Drops Off

Improved Flow:

User Starts Setup



Real-Time VPA Validation



Enter UPI PIN



Bank Timeout Detected



Smart Retry Engine Triggered



Retry Guidance / Reminder



Mandate Setup Success

4. Project Objectives

These are the outcomes I am trying to drive with the product changes recommended in this document:

#	Objective	How we measure it	Target
O-1	Recover failures caused by bank timeouts	Recovery rate of retried timeout failures	At least 50% of timeouts recovered
O-2	Raise the overall mandate success rate	Successful setups / total attempts	From 42.35% to ~52%
O-3	Cut down user input errors	Share of failures from PIN or VPA issues	Reduce by at least 20%
O-4	Make the failure experience less frustrating	CSAT / NPS score on the mandate flow	At least 4.0 out of 5.0
O-5	Give the team visibility into failure trends	Analytics dashboard availability	99.5% uptime

5. Scope

5.1 What is included

- Smart Retry Engine for bank timeout errors (both server-side detection and client-side UX during retry)
- Guided Retry Timing: in-app messaging after failure + push notification scheduling
- Real-time VPA format validation on the client side
- Improved PIN error messaging with remaining attempt counter

- Analytics dashboard: failure breakdown by hour and day, with heatmap
- Automated alerting for timeout rate spikes (Slack + PagerDuty)
- A/B testing framework for retry message variants

5.2 What is not included

- Any changes to NPCI's or the banks' infrastructure, this project only touches our product layer
- Biometric or device-based PIN authentication (could be a future phase)
- Mandate amendment or cancellation flows
- International payment methods or non-UPI rails

6. Stakeholders

In a real team, these are the people who would need to be involved:

Role	Why they are involved
Product Manager – Payments	Owns the mandate setup flow; prioritises and signs off on requirements
Engineering Lead – Payments	Designs and builds the retry engine and validation changes
UX Designer	Designs the guided retry banner, error messages, and notification copy
Data Analyst	Builds the analytics dashboard and validates success rate calculations
QA Lead	Writes test plans and signs off on acceptance criteria
Banking Partnerships	Checks that retry logic stays within bank API rate limits
Legal / Compliance	Makes sure nothing in the retry or notification flow breaks NPCI / RBI guidelines
End Users	The people actually setting up UPI Autopay, ultimately who we are trying to help

7. Data Analysis Findings

All findings below come from SQL analysis of the synthetic dataset. The analysis covered: user_id, mandate_attempt_datetime, day_of_week, hour_of_day, setup_status, and failure_reason.

7.1 Overall Funnel

Status	Count	Percentage
Success	847	42.35%
Failed	1,153	57.65%
Total	2,000	100.00%

More than half of all attempts fail. This is broadly in line with what RBI data suggests for the industry, and it confirms the problem is real and significant.

7.2 Success Rate by Hour of Day

The most striking finding was how much the time of day matters. Success rates are nearly double in the evening compared to the morning:

- 19:00–21:00: ~59% success rate (peak window)
- 06:00–10:00: ~26–30% success rate (worst window)
- Saturday 20:00: ~82% success — by far the best single slot observed

This strongly suggests that banking infrastructure congestion is a major driver of failures during morning hours. Evening hours, when load is lower, see far fewer timeouts.

7.3 Success Rate by Day of Week

Day	Success Rate	Notes
Monday – Friday	41.8% – 42.9%	Near average; weekday morning traffic drags down the daily rate
Saturday	44.22%	Best day overall; especially strong at 20:00 (~82%)
Sunday	43.6%	Above average; lower system load on weekends

7.4 Failure Reason Breakdown

Failure Reason	Estimated Count	% of Failures	Category
Bank Timeout	392	34%	System / Infrastructure

Failure Reason	Estimated Count	% of Failures	Category
PIN Error	381	33%	User Input
Invalid VPA	380	33%	User Input
Total	1,153	100%	

The split between system-side and user-side failures is almost 50/50, which means we need two different types of solution — one that quietly handles infrastructure problems, and one that helps users get their inputs right the first time.

8. Functional Requirements

Requirements are tagged Must Have, Should Have, or Could Have based on their impact on the core problem.

8.1 Smart Retry Engine

Req ID	Requirement	Priority
FR-001	System must detect failure_reason = 'Bank Timeout' and automatically retry within 30 seconds	Must Have
FR-002	Maximum 3 automatic retries with exponential backoff: 30s, 60s, 120s	Must Have
FR-003	System must NOT automatically retry PIN errors or Invalid VPA failures	Must Have
FR-004	Every retry attempt must be logged with user_id, attempt number, timestamp, and outcome	Must Have
FR-005	Retry intervals and max attempts must be configurable via admin panel without a code release	Should Have

8.2 Guided Retry Timing

Req ID	Requirement	Priority
FR-006	After any failed attempt, display an in-app banner with a suggested retry window based on 30-day rolling success data	Must Have
FR-007	Banner must include a 'Remind Me' button that schedules a push notification at the suggested time	Must Have
FR-008	Push notification must deep-link back to the mandate setup screen with pre-filled user fields intact	Must Have
FR-009	Push notification must not fire if the mandate has already been successfully set up	Must Have

Req ID	Requirement	Priority
FR-010	Suggested retry window must be dynamically computed from real hourly success data	Should Have
FR-011	System should support A/B testing of at least 2 banner message variants	Could Have

8.3 Input Validation & Error UX

Req ID	Requirement	Priority
FR-012	VPA field must validate format (xxx@yyy) client-side on blur and on submit	Must Have
FR-013	Invalid VPA format must show inline error: 'Please enter a valid UPI ID (e.g. name@upi)'	Must Have
FR-014	PIN error response must show remaining attempt count: 'Incorrect PIN. X attempts remaining.'	Must Have
FR-015	After the final PIN attempt, system must show a one-tap redirect to UPI PIN reset	Must Have
FR-016	PIN field should clear automatically after each failed attempt	Should Have
FR-017	An info tooltip explaining UPI Autopay in plain language should be available on the setup screen	Should Have

8.4 Analytics & Monitoring

Req ID	Requirement	Priority
FR-018	Analytics portal must provide a failure reason report filterable by date range, hour, and day of week	Must Have
FR-019	Report should include a heatmap of success rate by hour x day	Should Have
FR-020	System must alert automatically when bank timeout rate exceeds 40% in a rolling 15-minute window	Must Have
FR-021	Alerts must go to Slack and PagerDuty with: timeout rate, volume, timestamp, and affected bank	Must Have
FR-022	Analytics data must not be more than 1 hour behind actual transaction time	Must Have
FR-023	Reports should be exportable as CSV and PDF	Should Have

9. Non-Functional Requirements

Category	Requirement	Target
Performance	Time from failure detection to first retry initiation	Under 2 seconds
Performance	In-app banner display time after failure	Under 2 seconds
Availability	Retry engine uptime	99.9% monthly
Availability	Analytics dashboard uptime	99.5% monthly
Scalability	Retry engine capacity at peak mandate volume	Must support high transaction volumes during peak traffic periods
Security	UPI PIN must never be stored or logged by the retry engine	Mandatory
Compliance	Retry logic must stay within NPCI's mandate processing guidelines	Mandatory
Accessibility	Guided retry banner should follow accessibility best practices	Required
Data Retention	Retry attempt logs must be kept for 13 months per RBI guidelines	Mandatory

10. User Stories

Stories are grouped by Epic. Each one follows the format: As a [role], I want to [action] so that [outcome], with acceptance criteria listed below it.

Epic 1: Smart Retry Engine

Automatically detect and recover from transient bank timeout failures without requiring user action.

US-001 [High Priority]

As a *payment system*, I want to automatically detect bank timeout failures and retry the mandate request with a short delay so that it reduces permanent failures caused by temporary banking infrastructure issues.

Acceptance Criteria:

- System identifies failure_reason = 'Bank Timeout' and triggers retry within 30 seconds
- Maximum of 3 automatic retries per mandate attempt
- User sees a non-intrusive message like 'Connecting with your bank...' during retry
- Each retry attempt is logged with timestamp and attempt number
- System does not retry for user-input errors (PIN error, Invalid VPA)

US-002 [High Priority]

As a *payment system*, I want to use exponential backoff between retry attempts so that the system avoids overloading banking servers when they are already under stress.

Acceptance Criteria:

- First retry at 30 seconds, second at 60 seconds, third at 120 seconds
- Backoff intervals can be updated from an admin config without a code release
- User sees a countdown showing when the next retry will happen
- If all retries fail, the system moves into the Guided Retry Timing flow (US-004)

US-003 [Medium Priority]

As a *product analyst*, I want to view a dashboard showing retry success rates broken down by failure reason and time of day so that I can monitor how effectively the retry engine is working *and catch new patterns early*.

Acceptance Criteria:

- Dashboard shows retry success rate per failure_reason category
- Data refreshes every 15 minutes
- Can filter by date range, day of week, and hour of day
- Data is exportable to CSV

Epic 2: Guided Retry Timing

Nudge users toward the evening time windows where mandate success rates are significantly higher.

US-004 [High Priority]

As a *user who just had a failed mandate attempt*, I want to get a clear suggestion in the app about the best time to try again so that I can improve my chances of completing the mandate *without needing to figure out the right time myself*.

Acceptance Criteria:

- After any failed attempt, a banner appears within 2 seconds with suggested retry window (e.g. '7 PM – 9 PM')
- Banner copy reads: 'Mandate setup works best during evening hours. Try again tonight for faster approval.'
- User can dismiss the banner or tap 'Remind Me at 7 PM' to set a push notification
- The suggested window is calculated from the last 30 days of hourly success rate data
- On Saturday, the banner specifically calls out 8 PM as the best slot

US-005 [Medium Priority]

As a *user*, I want to receive a push notification at my chosen retry time that takes me straight back to the mandate setup screen so that I can complete the setup without needing to remember to return to the app later.

Acceptance Criteria:

- Notification arrives within 1 minute of the scheduled time
- Tapping it deep-links directly to the mandate setup screen with my previously entered details still filled in
- Notification text mentions the subscription or mandate context (e.g. app name)
- User can turn off retry reminders in notification settings
- Notification is not sent if the mandate was already set up successfully

US-006 [Low Priority]

As a *product manager*, I want to run A/B tests on different guided retry message variants so that I can identify which message variant drives the highest mandate completion rate.

Acceptance Criteria:

- System supports at least 2 concurrent message variants
- Variant assignment is randomised per user session
- Analytics tracks completion rate per variant
- A test can be stopped and a winner declared without a code deployment

Epic 3: Input Validation & Error UX

Catch user-input mistakes (bad VPA format, wrong PIN) early with clear, actionable feedback.

US-007 [High Priority]

As a *user*, I want to get instant feedback if I type my UPI ID in the wrong format so that I can correct my mistake before the request reaches the bank *and comes back as a hard failure*.

Acceptance Criteria:

- VPA field validates format (xxx@yyy) when I move to the next field and again on submit
- Inline error shows in red: 'Please enter a valid UPI ID (e.g. name@upi)'
- A green tick appears next to the field when the format looks correct
- Format check runs on the client side — no API call needed
- If the VPA format is fine but the bank can't find it, a separate message shows: 'UPI ID not found. Please check and retry.'

US-008 [High Priority]

As a *user*, I want to understand exactly what went wrong when I enter the wrong UPI PIN, and know how many tries I have left so that I can correct my mistake quickly *and not accidentally lock myself out without realising it*.

Acceptance Criteria:

- On PIN error, screen shows: 'Incorrect PIN. You have X attempts remaining before your UPI PIN is locked.'
- Attempt count is pulled from the bank's response and shown accurately
- After the last failed attempt, a one-tap option appears to reset the UPI PIN
- PIN field clears automatically after each failed attempt
- Error message can be dismissed and the retry button stays visible

US-009 [Medium Priority]

As a *user*, I want to see a short plain-language explanation of what a UPI Mandate actually is before I set one up so that I feel confident about what I am authorising *before I proceed*.

Acceptance Criteria:

- An info icon sits next to the 'Set Up UPI Autopay' heading
- Tapping it opens a bottom sheet with a simple explanation in under 60 words
- Bottom sheet includes a link to full terms and conditions
- The tooltip does not cover or block the main action button

Epic 4: Analytics & Monitoring

Give the team visibility into failure trends and alert them when something goes wrong.

US-010 [High Priority]

As a *data analyst*, I want to pull a failure reason report segmented by hour of day and day of week so that I can identify time-based failure patterns *and back up product and infra decisions with real data*.

Acceptance Criteria:

- Report is available in the analytics portal with date range, hour, and day filters
- Failure reasons are grouped as: Bank Timeout, PIN Error, Invalid VPA, Other
- A heatmap of success rate by hour x day is included
- Data is no more than 1 hour delayed from actual transaction time
- Report can be exported as CSV or PDF

US-011 [Medium Priority]

As a part of *engineering team*, I want to get an automatic alert when bank timeout failures spike above a set threshold so that the team can identify banking infrastructure issues early, *before they affect a large number of users*.

Acceptance Criteria:

- Alert fires when bank timeout rate exceeds 40% in any rolling 15-minute window
- Alert goes to Slack and PagerDuty with: timestamp, failure rate, volume, and affected bank(s)
- Threshold is configurable without a code deployment
- Alert auto-resolves once the rate stays below the threshold for 15 minutes

US-012 [Low Priority]

As a *product manager*, I want to see a monthly mandate funnel report that compares success rates before and after each product change so that I can measure and present the actual impact *of each improvement in a way that is easy to present*.

Acceptance Criteria:

- Report shows month-on-month success rate trend
- I can manually add change annotations (e.g. 'Smart Retry launched on 1 Mar')

- Projected vs actual success rate comparison is available
- Report is shareable via a view-only link

11. Impact Estimate

These projections assume 50% recovery of bank timeout failures through the Smart Retry Engine. They are meant to be directional — the real numbers should be validated through A/B testing after launch.

Metric	Before	After (Projected)	Change
Overall Mandate Success Rate	42.35%	~52%	+~10 percentage points
Bank Timeout Recovery	0% (no retry today)	At least 50%	+50 pp
User Input Error Share	~33% of failures	~26% of failures	~20% reduction
Subscription Activation Rate	Baseline	+8–12% relative	To be confirmed post-launch
Onboarding Drop-off	Baseline	Reduced	To be confirmed post-launch

12. Assumptions & Constraints

12.1 Assumptions

- The synthetic dataset reasonably reflects real production UPI failure distributions
- Banking infrastructure congestion is the main reason success rates are lower in the morning
- Users have push notifications enabled at the time of mandate setup
- Bank APIs support idempotent mandate retries so the same mandate is not created twice
- The retry engine will operate within NPCI's rate limits for mandate submissions

12.2 Constraints

- The retry engine cannot change how bank servers process mandates — it can only resubmit the request
- The platform must never store, cache, or re-use a user's UPI PIN between attempts
- Push notification delivery depends on the user's device settings and carrier reliability
- Any A/B test copy must still meet NPCI and RBI disclosure requirements for mandate authorisation
- All findings should be validated against live production data before finalising thresholds

13. Risks & Mitigations

ID	Risk	Likelihood	Impact	Mitigation
R-001	Bank API rejects a retry as a duplicate / replay attack	Medium	High	Ensure duplicate retries are safely handled in line with banking API guidelines; confirm approach with banking partners before build
R-002	Retry engine adds load to bank servers during an outage and makes things worse	Low	High	Cap global retry rate; use backoff limits; loop in Banking Partnerships before launch
R-003	Suggested retry timing becomes stale if success patterns change over time	Medium	Medium	Use a 30-day rolling window; add a staleness check so stale data triggers a refresh
R-004	Push notification deep link fails if the user has uninstalled the app	High	Low	Fall back gracefully to the web-based mandate setup URL
R-005	Analytics data latency creeps above 1 hour during high-volume periods	Low	Medium	Use near real-time analytics ingestion with latency monitoring and alerting

14. Dependencies

- Bank API documentation confirming support for idempotent mandate retries
- Legal / Compliance sign-off that retry and notification flows are within NPCI / RBI guidelines
- Mobile push notification integration support from the Mobile team
- Streaming data pipeline for analytics (dependency on the Data Engineering team)
- An A/B testing tool in place before the Guided Retry Timing feature launches
- Design System components for the in-app banner and bottom sheet (UX team deliverable)

15. Acceptance Criteria

The project can be considered done when all of the following are true in production:

1. Overall mandate success rate is at or above 52% over a 30-day measurement window after launch
2. Smart Retry Engine recovers at least 50% of bank timeout failures within the same session

3. All Must Have requirements (FR-001 to FR-023 where marked) are built and pass QA regression
4. Analytics dashboard is live with data latency under 1 hour and at least 99.5% availability
5. Automated alert fires correctly in staging when a simulated timeout spike is triggered
6. No new NPCI or RBI compliance issues are introduced by the retry or notification logic

16. Glossary

Term	Definition
UPI	Unified Payments Interface, India's real-time digital payments rail, operated by NPCI
UPI Autopay	NPCI's recurring payment mandate framework on UPI, enabling pre-authorised debits
Mandate	A user-authorised standing instruction to debit their bank account at defined intervals
VPA	Virtual Payment Address, a UPI identifier like name@bankname that maps to a bank account
UPI PIN	A 4–6 digit PIN used to authorise UPI transactions
Bank Timeout	A failure where the bank's server does not respond within the UPI rail's time limit
NPCI	National Payments Corporation of India, the body that runs and regulates UPI
RBI	Reserve Bank of India, India's central bank and payment system regulator
BRD	Business Requirements Document, a structured write-up of what a product initiative needs to achieve
TPS	Transactions Per Second, a measure of how many transactions a system can handle at once

— End of Document —